

# Documento E — Implementação dos Testes de Integração

**Projeto:** App de Estoque e Gestão Financeira (Sublimação)

**Tecnologia:** Flutter e Firebase

**Arquitetura:** MVVM

**Tipo de teste:** Integração / End-to-End (E2E)

## 1. Identificação do Arquivo de Teste

**Caminho do arquivo no repositório:**

integration\_test/inventory\_flow\_test.dart

Este arquivo centraliza a execução dos testes automatizados de interface e integração com o backend em nuvem do Firebase Firestore, mapeando as interações completas do usuário desde o preenchimento do formulário até a verificação de persistência e reatividade na listagem.

## 2. Ferramentas e Métodos Utilizados

- **integration\_test:** Biblioteca oficial do Flutter para conduzir testes de comportamento e fluxos completos em dispositivos ou emuladores.
- **flutter\_test:** Framework base para asserções (expect) e estruturas de grupos de testes.
- **WidgetTester:** Ferramenta programática usada para construir o ambiente do componente, injetar dados e capturar elementos da árvore de widgets.
- **pumpWidget:** Método responsável por inflar e renderizar o widget principal da aplicação ou da feature sob teste.
- **pumpAndSettle:** Método crítico para aguardar a conclusão de todas as animações, transições de estado e respostas assíncronas de rede pendentes (Firebase Streams).
- **enterText:** Simulação de digitação do usuário nos campos textuais do formulário de cadastro.
- **tap:** Simulação de cliques em botões de ação, como salvar ou incrementar quantidades.
- **find:** Mecanismo de busca para localizar elementos visuais na interface por meio de chaves (Keys), textos ou tipos de componentes.

## 3. Objetivo da Implementação

Validar o funcionamento integrado real entre a camada de interface (InventoryPage), a gerência de estado no InventoryViewModel, as validações de restrições de negócio locais e a comunicação assíncrona com as coleções do Firebase Firestore, descartando o uso de dados simulados ou mockados.

## 4. Estrutura do Código de Teste Implementado

Abaixo está detalhada a codificação estrutural aplicada para cobrir as especificações de testes estabelecidas:

```
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';
import 'package:projeto_pi_mobile/main.dart' as app;

void main() {
  // Inicialização obrigatória do vínculo para execução de testes de integração reais
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  group('Testes de Integração Ponta a Ponta - Módulo de Estoque (Inventory)', () {

    testWidgets('TC01 - Cadastro de insumo completo e válido com persistência',
      (WidgetTester tester) async {
        // Inicializa a aplicação conectada ao Firebase de teste
        app.main();
        await tester.pumpAndSettle();

        // Navega até a tela de inventário/estoque
        final Finder inventoryTab = find.byKey(const Key('inventory_nav_btn'));
        await tester.tap(inventoryTab);
        await tester.pumpAndSettle();

        // Toca no botão para abrir o formulário de novo produto
        final Finder addProductFab = find.byKey(const Key('add_product_fab'));
        await tester.tap(addProductFab);
        await tester.pumpAndSettle();

        // Injeta os dados válidos respeitando os 6 atributos obrigatórios do RF01
        await tester.enterText(find.byKey(const Key('input_nome')), 'Caneca de Cerâmica Branca');
        await tester.enterText(find.byKey(const Key('input_categoria')), 'Sublimação');
        await tester.enterText(find.byKey(const Key('input_qtd_atual')), '100');
        await tester.enterText(find.byKey(const Key('input_qtd_minima')), '10');
        await tester.enterText(find.byKey(const Key('input_preco_compra')), '12.50');
        await tester.enterText(find.byKey(const Key('input_preco_venda')), '35.00');
        await tester.pumpAndSettle();

        // Dispara a ação de salvamento
        final Finder saveButton = find.byKey(const Key('btn_salvar_produto'));
        await tester.tap(saveButton);

        // Aguarda o processamento da requisição de rede e retorno da navegação
        await tester.pumpAndSettle();

        // Assertão: Verifica se o produto foi salvo e consta na listagem atualizada via Stream
        expect(find.text('Caneca de Cerâmica Branca'), findsOneWidget);
      });

    testWidgets('TC02 - Cadastro com omissão de campos obrigatórios (Bloqueio Local)',
```

```

(WidgetTester tester) async {
  app.main();
  await tester.pumpAndSettle();

  // Navega e abre o formulário
  await tester.tap(find.byKey(const Key('inventory_nav_btn')));
  await tester.pumpAndSettle();
  await tester.tap(find.byKey(const Key('add_product_fab')));
  await tester.pumpAndSettle();

  // Preenche apenas alguns campos, deixando outros obrigatórios vazios
  await tester.enterText(find.byKey(const Key('input_nome')), 'Camiseta Poliéster');
  await tester.enterText(find.byKey(const Key('input_categoria')), 'Vestuário');
  await tester.enterText(find.byKey(const Key('input_qtd_atual')), '50');
  // Quantidade mínima, preço de compra e preço de venda deixados em branco
  await tester.pumpAndSettle();

  // Tenta salvar o formulário incompleto
  await tester.tap(find.byKey(const Key('btn_salvar_produto')));
  await tester.pumpAndSettle();

  // Asserção: O app deve exibir a mensagem de erro em um SnackBar ou componente de
  alerta
  expect(find.text('Preencha todos os campos obrigatórios'), findsOneWidget);
});

testWidgets('TC03 - Baixa de estoque além do limite (Impedir Estoque Negativo)',
(WidgetTester tester) async {
  app.main();
  await tester.pumpAndSettle();

  // Acessa a página de inventário
  await tester.tap(find.byKey(const Key('inventory_nav_btn')));
  await tester.pumpAndSettle();

  // Tenta acionar uma redução de estoque drástica usando botões de ajuste rápido
  // Supondo um item com quantidade menor do que o decremento solicitado
  final Finder decrementBtn = find.byKey(const Key('btn_decrementar_lote')).first;
  await tester.tap(decrementBtn);
  await tester.pumpAndSettle();

  // Asserção: Garante o bloqueio visual e impede atualização inválida
  expect(find.text('Estoque insuficiente'), findsOneWidget);
});
});
}

```

## 5. Cenários Cobertos e Fluxo de Execução

O script implementado cobre de ponta a ponta as interações críticas do aplicativo, agindo como um usuário físico que preenche as inputs e clica nos botões. Através do uso extensivo de `pumpAndSettle`, a automação respeita a natureza assíncrona do Firebase, garantindo que o teste não avance até que o documento em nuvem seja totalmente processado e sincronizado com a árvore de componentes visuais do Flutter.